

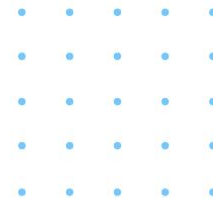
Mạng rơ non hồi quy

Recurrent Neural Network



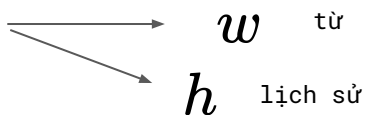
Mô hình ngôn ngữ

Language Models



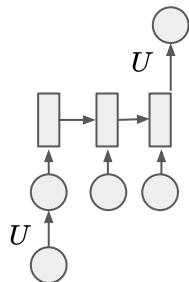
Mô hình ngôn ngữ (**Language Model**) được thiết kế với mục tiêu đo lường **phân phối xác suất** của các đơn vị ngôn ngữ (từ, chữ).

Nhiệm vụ: Tính $P(w|h)$ xác suất của một từ khi cho các từ trước đó (lịch sử).

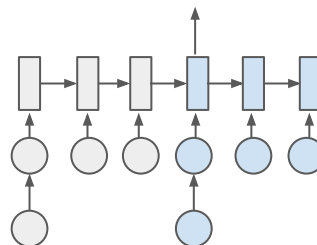


$P(\text{"ơ"} \mid \text{Việt nam đẹp lắm ta"}) = ???$

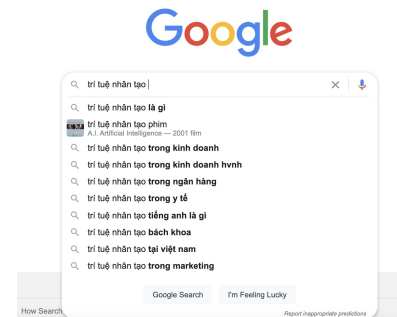
Language Model xuất hiện trong nhiều hệ thống khác nhau: đề xuất từ tiếp theo, máy dịch, sửa lỗi.



Mô hình dự đoán từ tiếp theo khi cho lịch sử

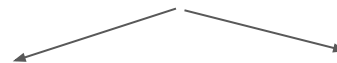
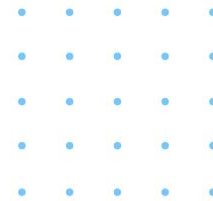


Mô hình dịch máy



N-Grams

N-Grams



$P(\text{vui} | \text{Học Tiếng Việt rất}) = ?$

$P(\text{dị} | \text{Học Tiếng Việt rất}) = ?$

Một n-gram là một nhóm các từ **liên tiếp nhau**.

Unigram



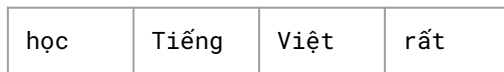
Bigram



Trigram

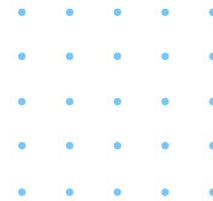


4-gram



N-Grams Language Model

N-Grams Language Model



n-1 từ
trước nó

Giả thiết Markov (Markov Assumption): Xác suất xảy ra một từ khi cho lịch sử sẽ bằng xác suất xảy ra từ đó khi cho **n-1 từ trước nó**

$P(\text{vui} | \text{Thông qua khảo sát của những tổ chức nghiên cứu, học Tiếng Việt rất}) = P(\text{vui} | \text{học Tiếng Việt rất})$

$$= \frac{P(\text{học Tiếng Việt rất vui})}{P(\text{học Tiếng Việt rất})} \approx \frac{C(\text{học Tiếng Việt rất vui})}{C(\text{học Tiếng Việt rất})}$$

Đếm số lượng cụm từ này trong các văn bản
C - count

E.g.

học Tiếng Việt rất vui

học Tiếng Việt rất vui

học Tiếng Việt hay lắm

học Tiếng Việt rất chán

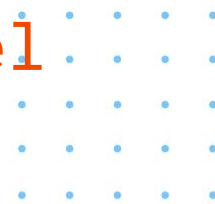
$C(\text{học Tiếng Việt rất vui}) = 2$

$C(\text{học Tiếng Việt rất}) = 3$

$P(\text{vui} | \text{học Tiếng Việt rất}) = 2/3$

Điểm yếu của N-Grams Language Model

N-Grams Language Model Weakness



$$P(\text{vui} | \text{học Tiếng Việt rất}) = \frac{C(\text{học Tiếng Việt rất vui})}{C(\text{học Tiếng Việt rất})}$$



Nếu $C(\text{học Tiếng Việt rất vui}) = 0$, trong câu **không tồn tại** “học Tiếng Việt rất vui” dẫn đến xác suất trên bằng 0.

Solution

Sử dụng **smoothing/discounting**

- ☐ Laplace (add-one) smoothing
- ☐ add-k smoothing
- ☐ Kneser-Ney smoothing

Nếu $C(\text{học Tiếng Việt rất}) = 0$, trong câu **không tồn tại** “học Tiếng Việt rất” dẫn đến xác suất trên không tồn tại.

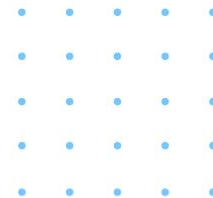
Solution

- ☐ stupid backoff



Backoff

Backoff



Nếu $C(\text{học Tiếng Việt rất vui}) = 0$ thì có thể sử dụng $C(\text{học Tiếng Việt rất})$

Nếu $C(\text{học Tiếng Việt rất}) = 0$ thì có thể sử dụng $C(\text{học Tiếng Việt})$

Trong **trường hợp xấu nhất** sẽ sử dụng unigrams $C(\text{học})$

Ưu điểm: đôi khi việc học này giúp mô hình học tốt hơn những **ngữ cảnh ít** được nhắc đến.



Interpolation

Interpolation

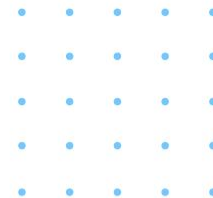
Đối lập với backoff - Interpolation đánh trọng số cho các grams và tổng hợp lại.

$$\hat{P}(w_n | w_{n-2} w_{n-1}) = \lambda_1 P(w_n | w_{n-2} w_{n-1}) + \lambda_2 P(w_n | w_{n-1}) + \lambda_3 P(w_n)$$

với $\sum_{i=1} \lambda_i = 1$

E.g.

$$\hat{P}(\text{học} | \text{Tôi đi}) = \lambda_1 P(\text{học} | \text{Tôi đi}) + \lambda_2 P(\text{học} | \text{đi}) + \lambda_3 P(\text{học})$$



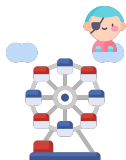
Điểm yếu của N-Grams Language Model

N-Grams Language Model Weakness

- Trong các văn bản **tồn tại** “học Tiếng Anh rất vui”, tuy nhiên khi đầu vào là “học Tiếng Việt rất” thì mô hình sẽ **không thể sinh ra** được từ “vui” mặc dù “Tiếng Anh” và “Tiếng Việt” có sự tương đồng về ngữ nghĩa (semantic similarity) và ngữ pháp (syntactic similarity)
- Trong một số câu có sự **phụ thuộc xa (distant dependencies)**, nếu chỉ quan tâm đến **1-3 từ trước đó** thì chưa đủ để dự đoán chính xác từ tiếp theo. E.g.

Anh ấy tên Ngọc rất thích trèo cây, đánh đu, chơi piano và mặc -----??

Có thể $P(\text{váy} | \text{chơi piano và mặc}) > P(\text{vest} | \text{chơi piano và mặc})$ vì mô hình đã bỏ qua từ **Anh** ấy.



Mục tiêu phải có một language model có khả năng từ:

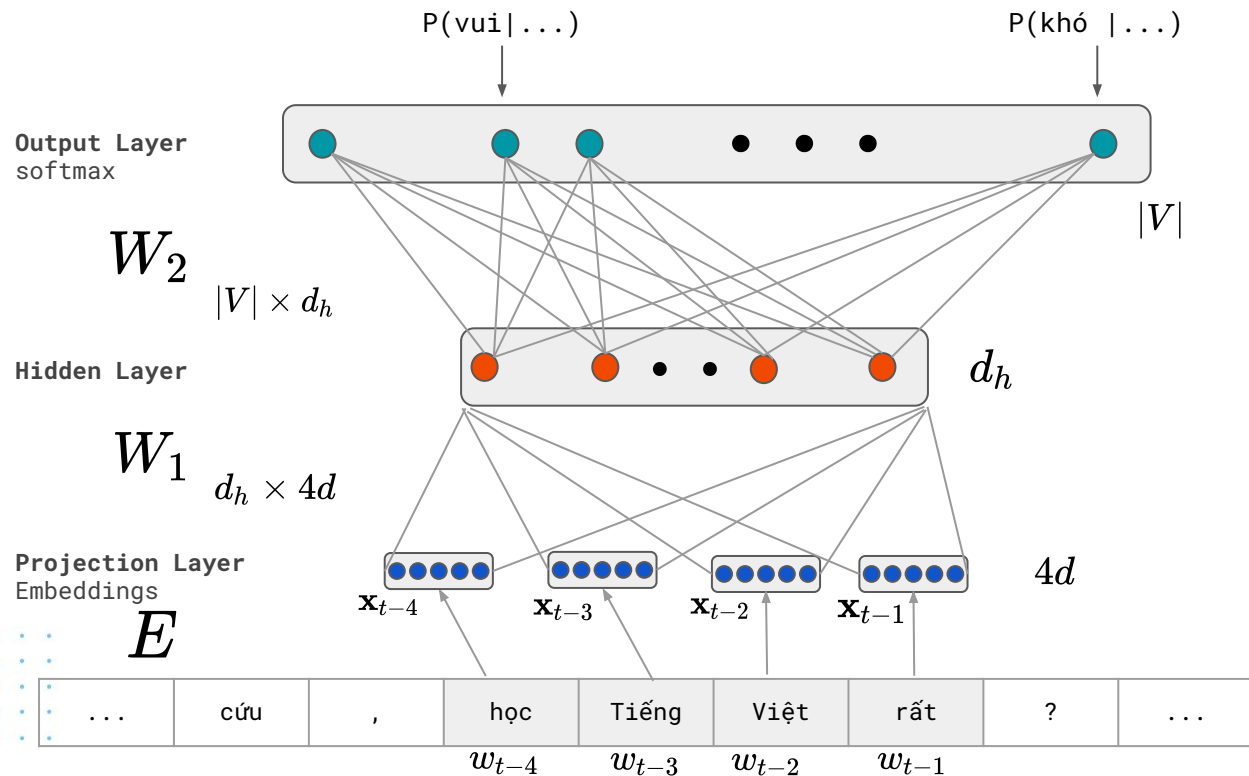
Học Tiếng Việt rất vui $\longrightarrow$ Học Tiếng Việt thật vui
Học Tiếng Anh rất vui
Học Tiếng Anh thật thú vị
Luyện tập Tiếng Anh rất hấp dẫn

Neural Language Models

Neural Language Model

<https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf>

Đây chính là ý tưởng đầu tiên từ **Bengio** training Word Embeddings. Sau này **Mikolov** đơn giản hoá bằng việc bỏ đi lớp Hidden Layer.



○ Sau đó lớp softmax cuối cùng sẽ tính xác suất xảy ra một từ khi cho lịch sử.

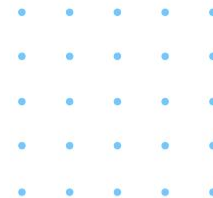
○ Đưa vector này qua một lớp Hidden Layer

○ Chuyển các từ thành Vector Embedding, sau đó nối các vector này lại với nhau



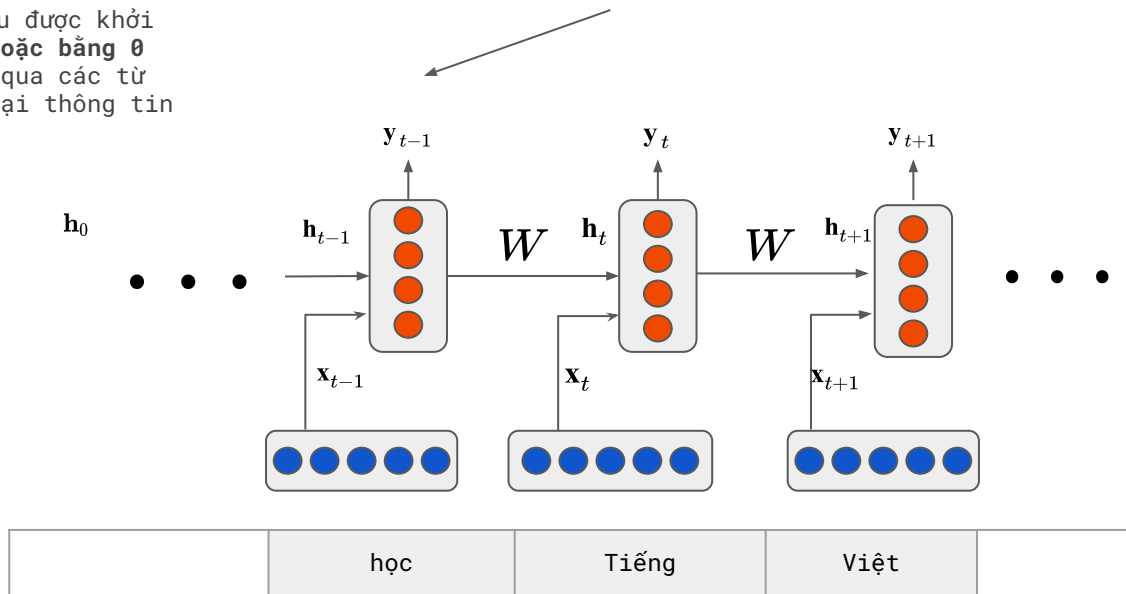
Mạng nơ ron hồi quy

Recurrent neural network



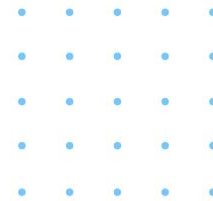
Ở mỗi bước (hay còn gọi là **timestep**), mô hình sẽ **bổ sung** mỗi quan hệ từ hiện tại vào trong lịch sử.

Vector h_0 lúc đầu được khởi tạo **ngẫu nhiên hoặc bằng 0** với mục đích đi qua các từ của câu và lưu lại thông tin từ **theo thứ tự**.



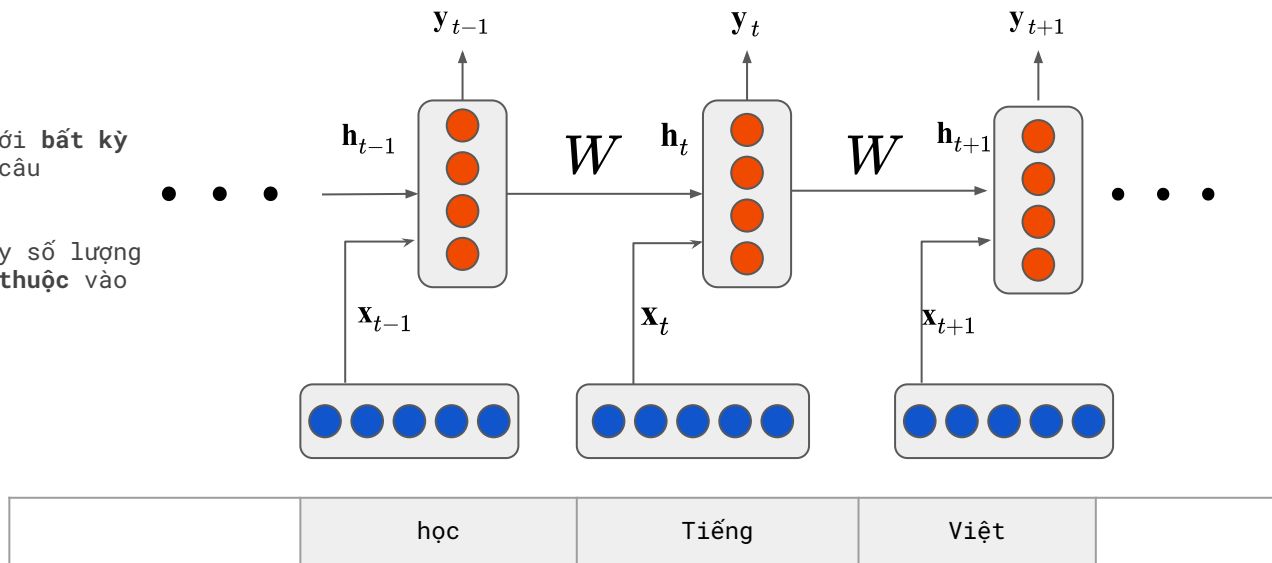
Ưu điểm của RNN

RNN Advantages



Ưu điểm

1. Có thể hoạt động với **bất kỳ** chiều dài nào của câu
2. Kích cỡ mô hình hay số lượng tham số **không phụ thuộc** vào chiều dài của câu

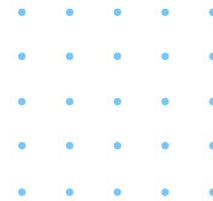


Mỗi một bước (time step) đều sử dụng duy nhất một bộ W và một vector h



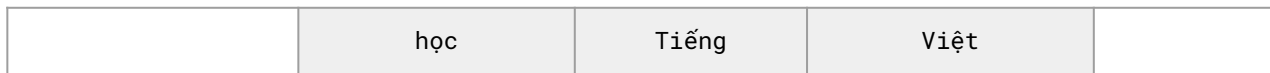
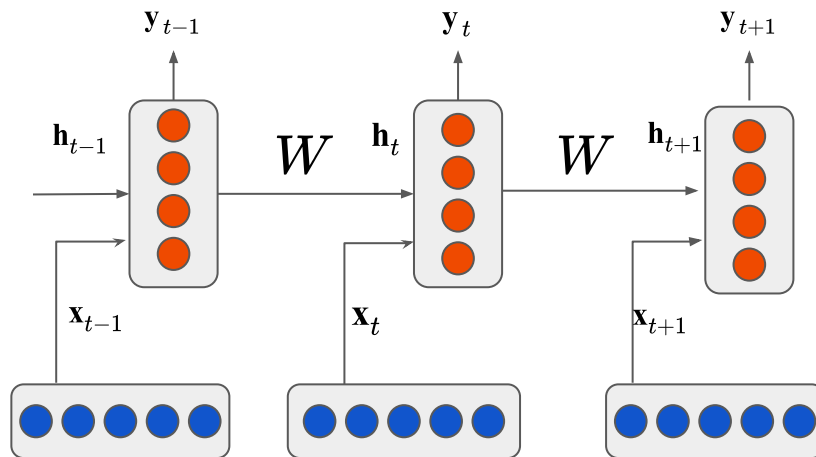
Nhược điểm của RNN

RNN Disadvantages



Nhược điểm

1. Việc tính toán yêu cầu tuần tự vì bước phía sau yêu cầu thông tin bước phía trước. Quá trình học **chậm**.
2. Đạo hàm rất dễ **bùng nổ** (Exploding Gradient) hoặc **biến mất** (Vanishing Gradient)



Mỗi một bước (time step) đều sử dụng duy nhất một bộ W và một vector h

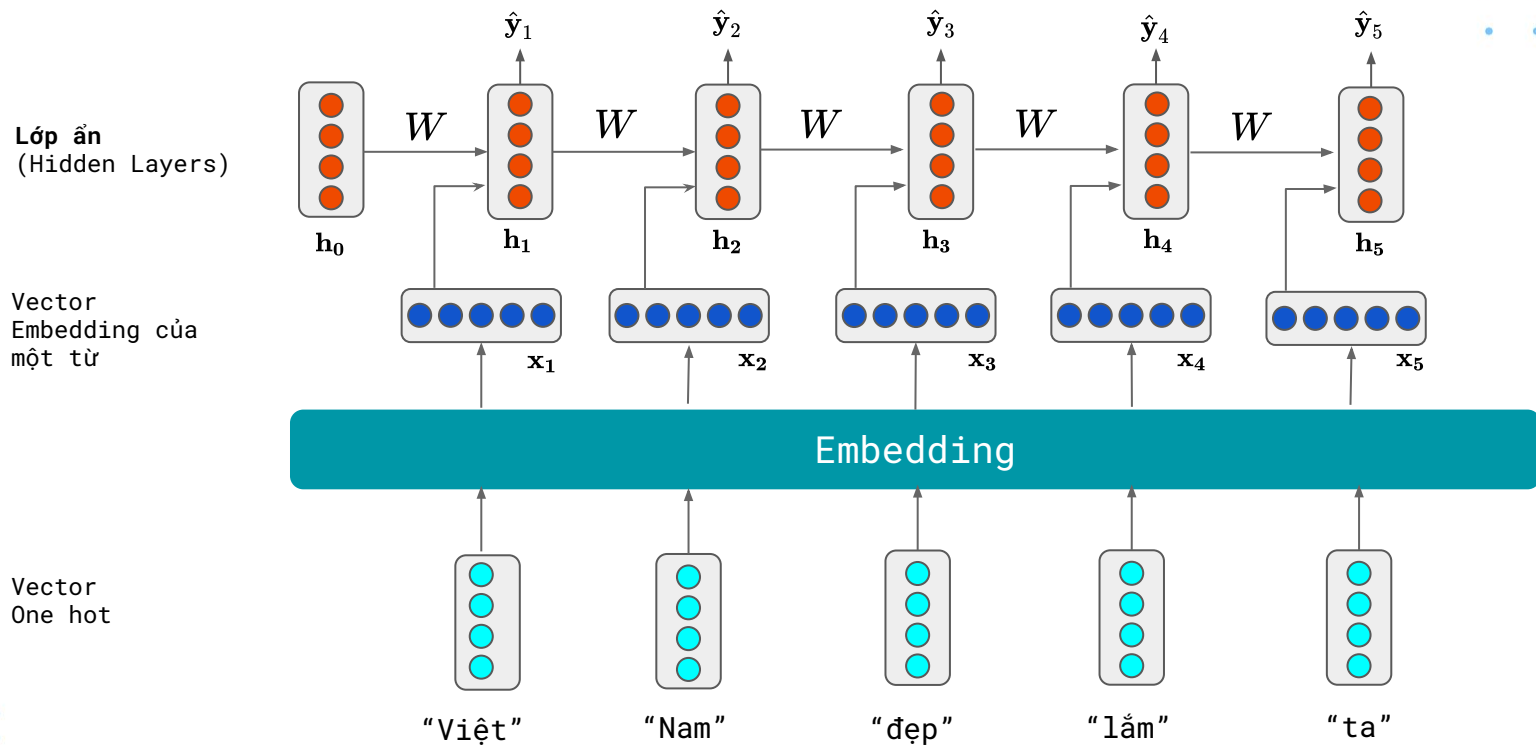


Sử dụng RNN là một mô hình ngôn ngữ

RNN-based language model

Mục tiêu dự đoán từ tiếp theo khi cho lịch sử.

“oi” ?



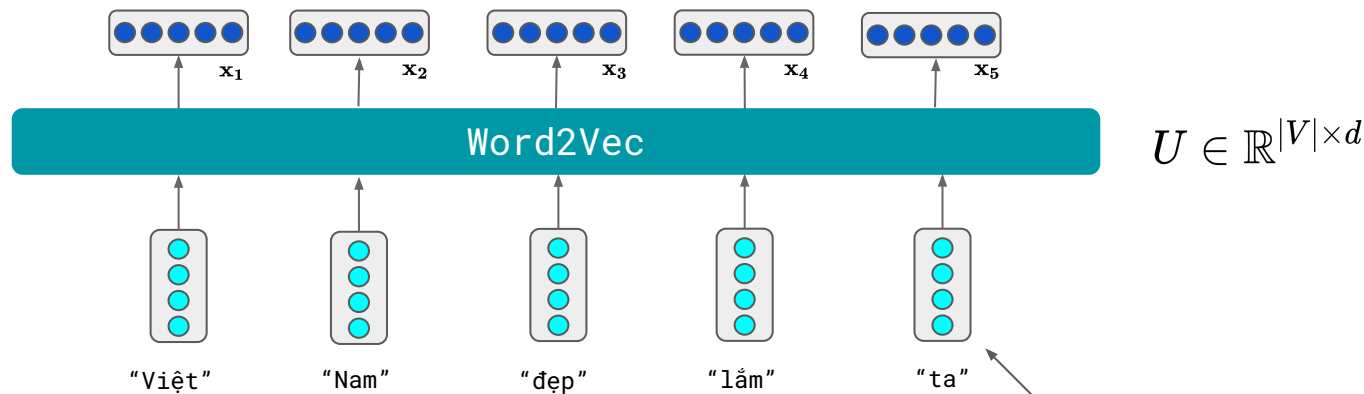
$$\hat{y}_1 = P(x_2 | \text{“Việt”})$$

$$\hat{y}_5 = P(x_6 | \text{“Việt Nam đẹp lắm ta”})$$

Mỗi một bước (time step) đều sử dụng duy nhất một bộ W và một vector h

Sử dụng Word2Vec

Word2Vec

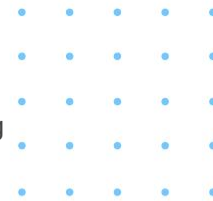


$|V|$ chiều từ điển

d chiều vector Embedding

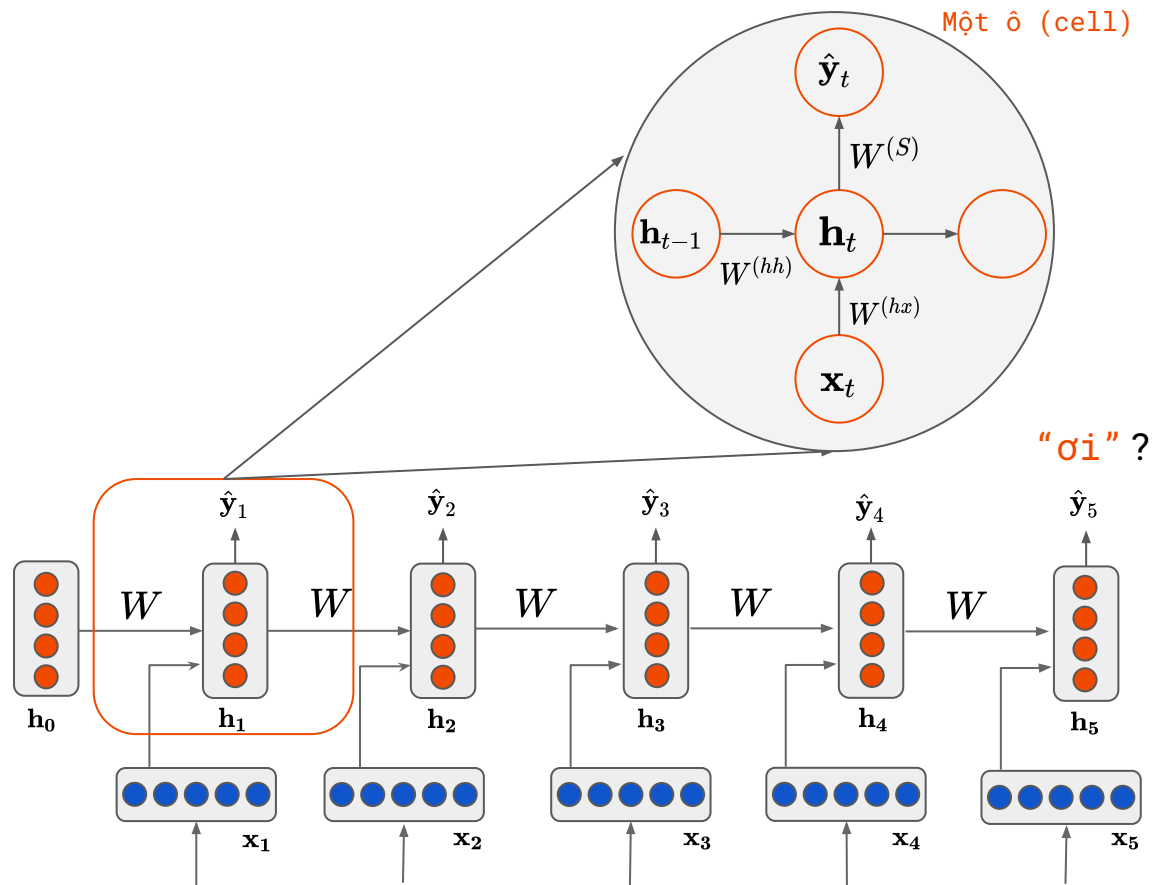
Vector one - hot
của một từ

$\mathbf{a} \in \mathbb{R}^{|V|}$



Mạng nơ ron hồi quy

Recurrent neural network



$$\mathbf{h}_t = \sigma(W^{(hh)}\mathbf{h}_{t-1} + W^{(hx)}\mathbf{x}_t)$$

$$\hat{\mathbf{y}}_t = \text{softmax}(W^{(S)}\mathbf{h}_t)$$

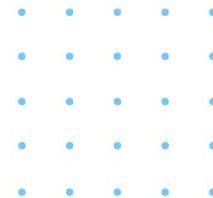
Xác suất **xây ra** một từ bất kỳ trong từ điển khi cho lịch sử

$$P(x_{t+1}=v_j|x_t,\dots,x_1) = \hat{\mathbf{y}}_{t,j}$$

Chú ý: Sẽ chỉ có **duy nhất một bộ W** và **một lịch sử h** dùng trong suốt quá trình training

Bài tập tính kích cỡ

Shape Calculation Exercise



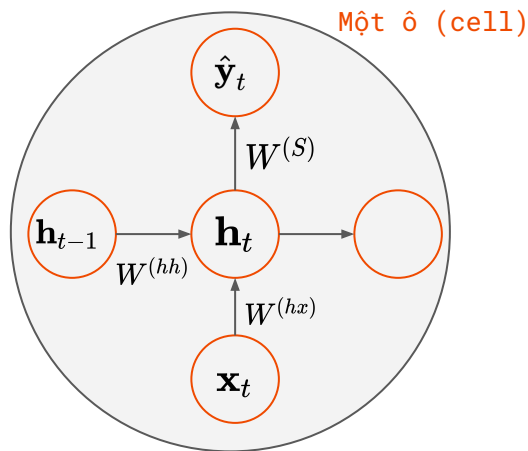
$$\mathbf{x} \in \mathbb{R}^{64}$$

$$W^{(hh)} \in ???$$

$$W^{(hx)} \in ???$$

$$\hat{\mathbf{y}} \in ???$$

$$\mathbf{h}_t \in \mathbb{R}^{32}$$



$$\mathbf{h}_t = \sigma(W^{(hh)}\mathbf{h}_{t-1} + W^{(hx)}\mathbf{x}_t)$$

$$\hat{\mathbf{y}}_t = \text{softmax}(W^{(S)}\mathbf{h}_t)$$

$$P_{(x_{t+1}=v_j|x_t,\dots,x_1)} = \hat{\mathbf{y}}_{t,j}$$

$\hat{\mathbf{y}}$

Vector **biểu diễn xác suất** trên toàn bộ từ điển từ có thể xảy ra khi cho lịch sử h

$|V|$ chiều từ điển (20000 từ)

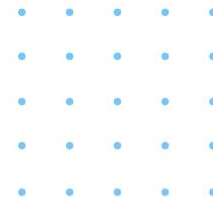
D_h chiều vector lịch sử h (32)

d chiều vector Embedding (64)



[Solution] Bài tập tính kích cỡ

[Solution] Shape Calculation Exercise



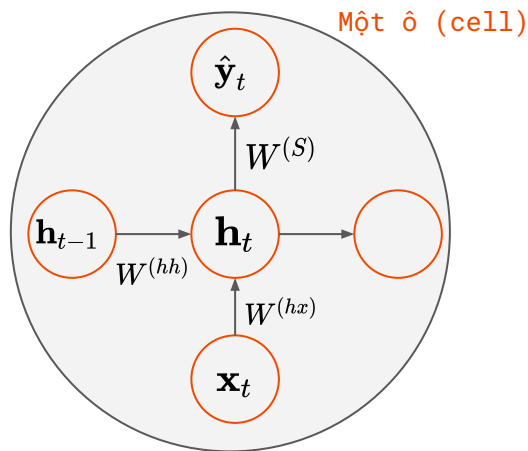
$$\mathbf{x} \in \mathbb{R}^d$$

$$W^{(hh)} \in \mathbb{R}^{D_h \times D_h}$$

$$W^{(hx)} \in \mathbb{R}^{D_h \times d}$$

$$\hat{\mathbf{y}} \in \mathbb{R}^{|V|}$$

$$\mathbf{h}_t \in \mathbb{R}^{D_h}$$



$$\mathbf{h}_t = \sigma(W^{(hh)}\mathbf{h}_{t-1} + W^{(hx)}\mathbf{x}_t)$$

$$\hat{\mathbf{y}}_t = \text{softmax}(W^{(S)}\mathbf{h}_t)$$

$$P(x_{t+1}=v_j|x_t,\dots,x_1) = \hat{\mathbf{y}}_{t,j}$$

$\hat{\mathbf{y}}$

Vector **biểu diễn xác suất** trên toàn bộ từ điển từ có thể xảy ra khi cho lịch sử h

$|V|$ chiều từ điển

D_h chiều vector lịch sử h

d chiều vector Embedding



Hàm mất mát sử dụng Teacher Forcing

Loss Function using Teacher Forcing

Training Progress

Hàm mất mát tại bước t

Hàm mất mát trên câu dài T

$$J^{(t)}(W) = - \sum_{j=1}^{|V|} y_{t,j} \log(\hat{y}_{t,j})$$

$$J = -\frac{1}{T} \sum_{t=1}^T \sum_{j=1}^{|V|} y_{t,j} \log(\hat{y}_{t,j})$$

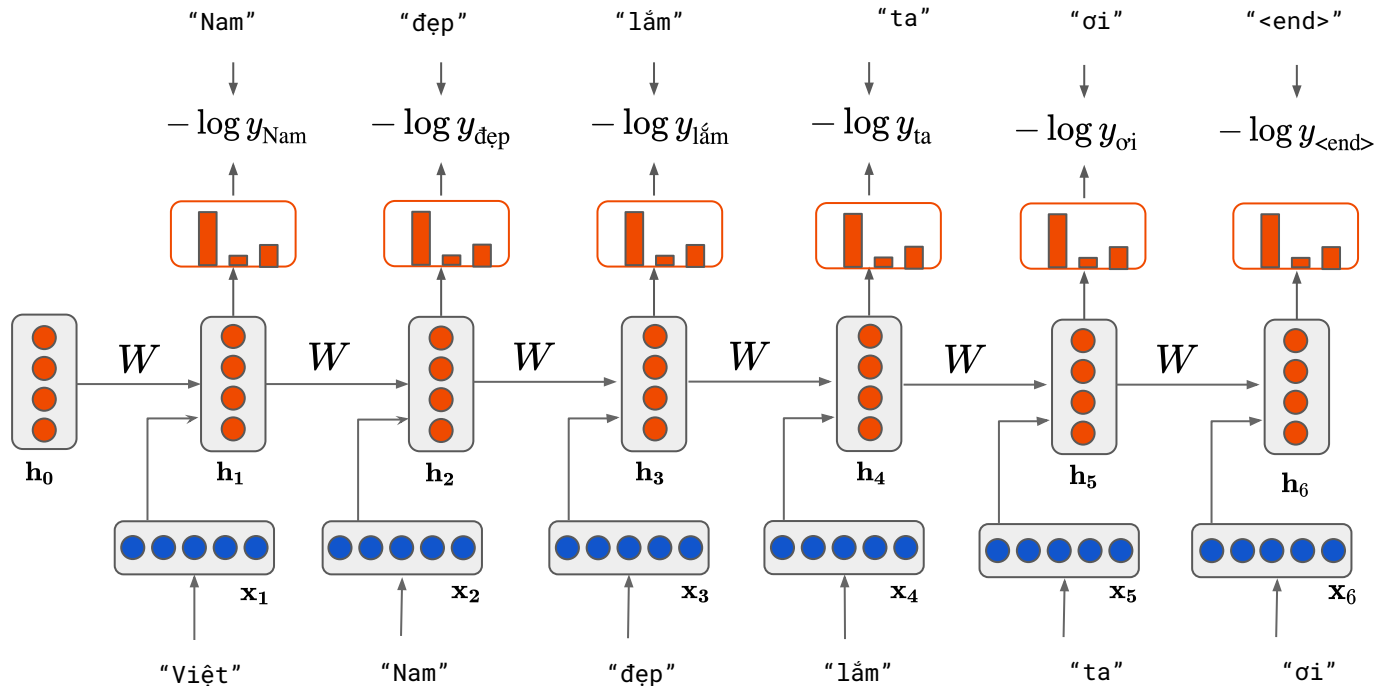
Label

Loss

Softmax

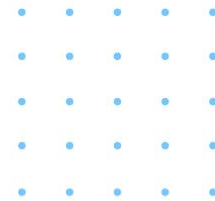
Lớp ẩn
(Hidden Layers)

Vector
Embedding của
một từ



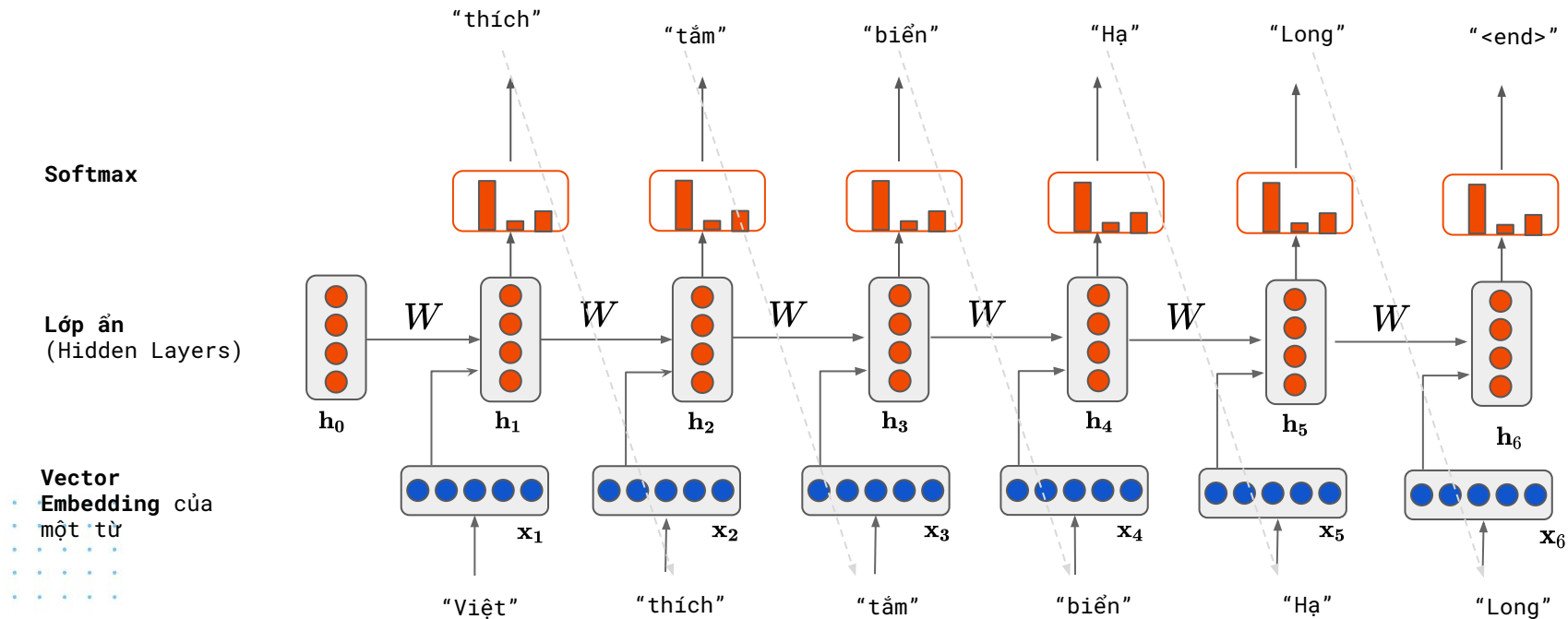
Quá trình Inference

Inference Progress



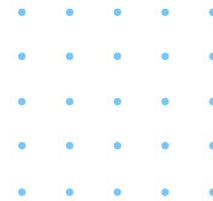
Inference Progress

- Đưa từ đầu tiên qua mô hình để dự đoán từ tiếp theo
- Tiếp tục quy trình này cho đến khi gặp token <end> hoặc câu dự đoán đạt đến độ dài cho phép. Đây là một **autoregressive language model**.



Vấn đề của RNN và Cách cải thiện

RNN Problems and Solutions



Problems

1 Hiện tượng Gradient Exploding

Hiện tượng Đạo hàm bùng nổ và vượt qua khả năng lưu trữ/ tính toán của máy tính

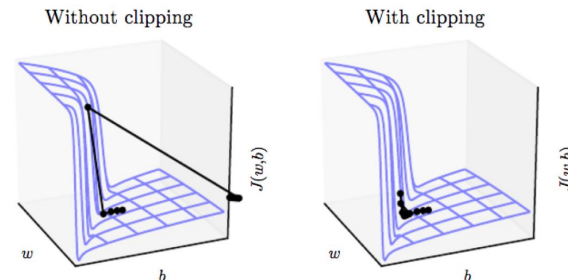
2 Hiện tượng Gradient Vanishing

Hiện tượng Đạo hàm triệt tiêu về 0, mô hình không học được thông tin.

3 Việc tính toán yêu cầu **tuần tự**, chưa thực hiện được song song

Solution

Áp dụng cắt Gradient (Gradient Clipping)



$$\hat{g} \leftarrow \frac{\partial J}{\partial W}$$

if $\|\hat{g}\| \geq threshold$ **then**

$$\hat{g} \leftarrow \frac{threshold}{\|\hat{g}\|} \hat{g}$$

end if

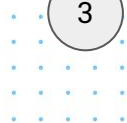
Thay đổi cách tính toán trong ô bằng GRU hoặc LSTM

Sử dụng mô hình **Transformer** với cơ chế **Attention**

<http://jalammar.github.io/illustrated-transformer/>

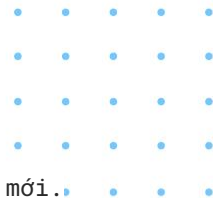
<https://arxiv.org/pdf/1810.04805.pdf>

<https://github.com/google-research/bert>



Perplexity (PP)

Perplexity (PP)



Cách kiểm tra hiệu năng của một mô hình ngôn ngữ là **tính toán khả năng sinh từ** trên một tập test mới.

1

Perplexity của một Language Model trên một **tập test** là xác suất nghịch đảo (**inverse probability**), và được chuẩn hoá (normalized) trên số lượng từ.

Giả sử có một tập test: $W = w_1 w_2 \dots w_n$

Tôi đi học

$$PP(W) = P(w_1 w_2 \dots w_n)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_n)}}$$

Sử dụng Chain Rules

$$\begin{aligned} &P(w_1 w_2 \dots w_n) \\ &= P(w_1)P(w_2|w_1)P(w_3|w_1 w_2) \dots P(w_n|w_1 \dots w_{n-1}) \\ &= \prod_{i=1}^n P(w_i|w_1 \dots w_{i-1}) \end{aligned}$$

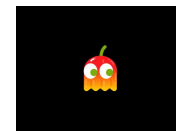
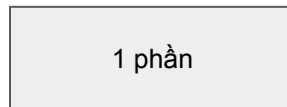
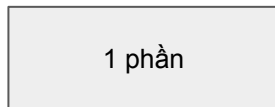
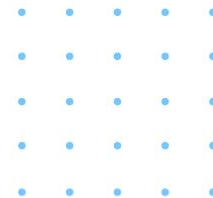
Kết luận

$$PP(W) = \sqrt[N]{\prod_{i=1}^n \frac{1}{P(w_i|w_1 \dots w_{i-1})}}$$

2

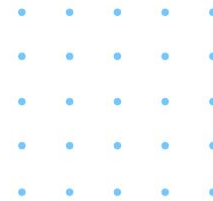
Tính chất: Perplexity **càng nhỏ** thì model **càng tốt**.





Long Short Term Memory (LSTM)

Long Short Term Memory (LSTM)



LSTM

$$f_t = \sigma(W_f x_t + U_f h_{t-1})$$

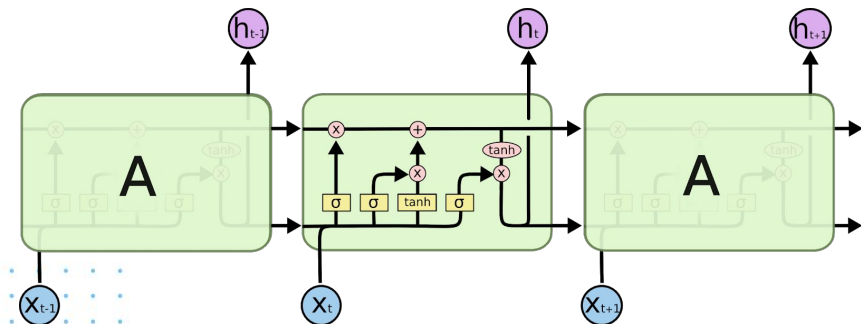
$$i_t = \sigma(W_i x_t + U_i h_{t-1})$$

$$\tilde{C}_t = \tanh(W_C x_t + U_C h_{t-1})$$

$$C_t = f_t \circ C_{t-1} + i_t \circ \tilde{C}_t$$

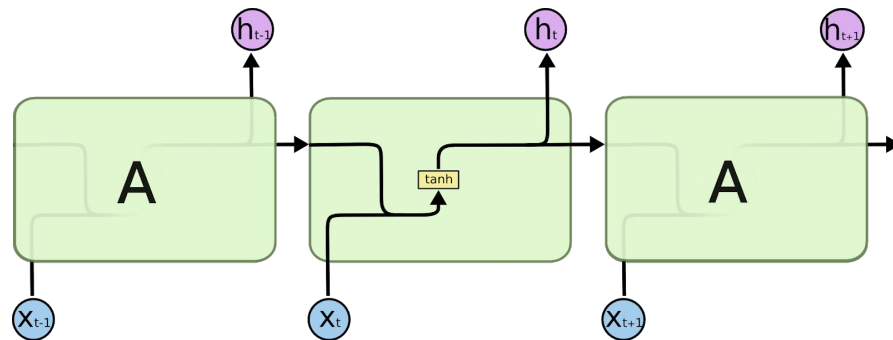
$$o_t = \sigma(W_o x_t + U_o h_{t-1})$$

$$h_t = o_t \circ \tanh(C_t)$$



Simple RNN

$$h_t = \sigma(Wx_t + Uh_{t-1})$$

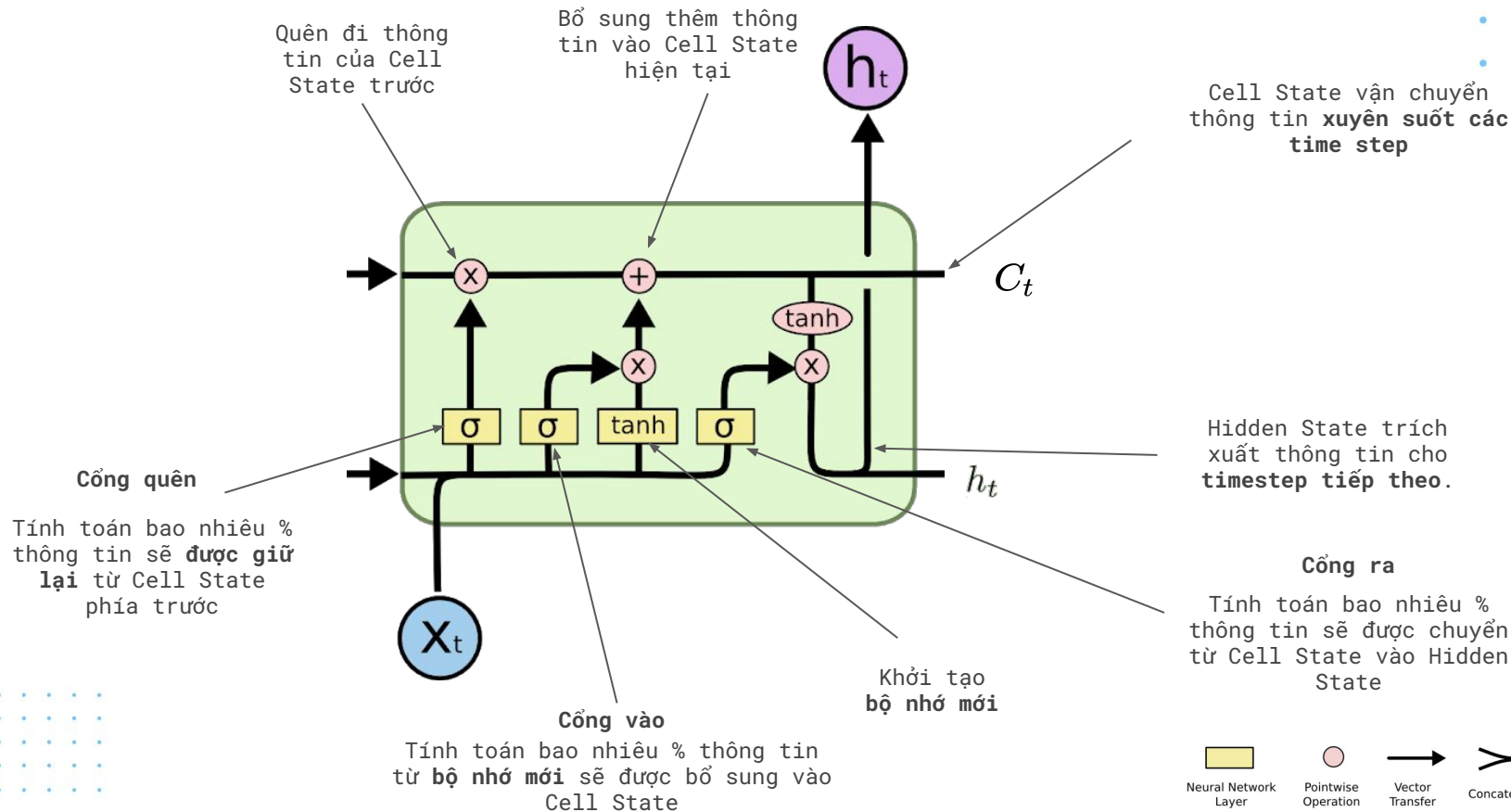


LSTM **tăng cường khả năng** linh hoạt trong lưu trữ thông tin cho các cell của RNN.

LSTM có khả năng cập nhật thông tin mới, giải phóng thông tin không còn cần thiết thông qua cơ chế sử dụng các cổng (gate).

Long Short Term Memory (LSTM)

Long Short Term Memory (LSTM)



Long Short Term Memory (LSTM)

Long Short Term Memory (LSTM)

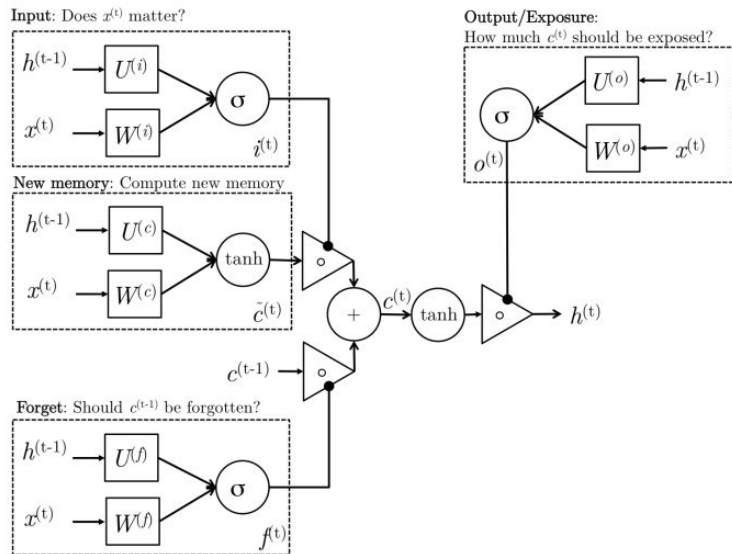
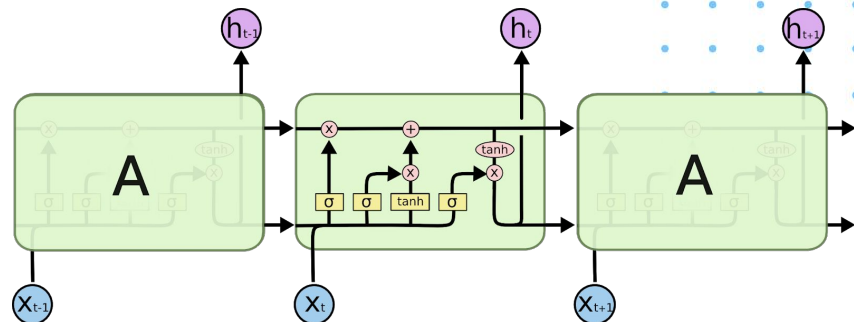


Figure 13: The detailed internals of a LSTM



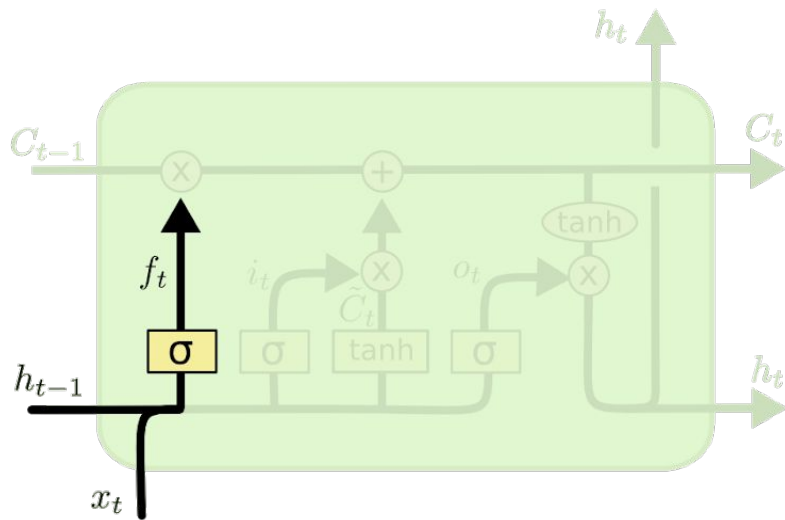
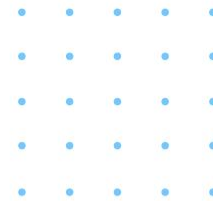
$$\begin{aligned}
 f_t &= \sigma(W_f x_t + U_f h_{t-1}) \\
 i_t &= \sigma(W_i x_t + U_i h_{t-1}) \\
 \tilde{C}_t &= \tanh(W_C x_t + U_C h_{t-1}) \\
 C_t &= f_t \circ C_{t-1} + i_t \circ \tilde{C}_t \\
 o_t &= \sigma(W_o x_t + U_o h_{t-1}) \\
 h_t &= o_t \circ \tanh(C_t)
 \end{aligned}$$

Forget Gate
Input Gate
New Memory Cell
Final Memory Cell
Output/Explode Gate

Bao nhiêu % dùng thông tin cũ +
Bao nhiêu % dùng thông tin mới

Mặc dù ô LSTM cải thiện được vấn đề Gradient Vanishing nhưng không triệt để

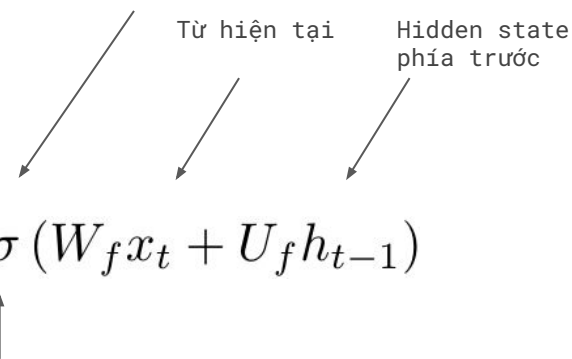
<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>



Hàm sigmoid:

Giá trị nằm trong đoạn $[0,1]$

$$f_t = \sigma(W_f x_t + U_f h_{t-1})$$

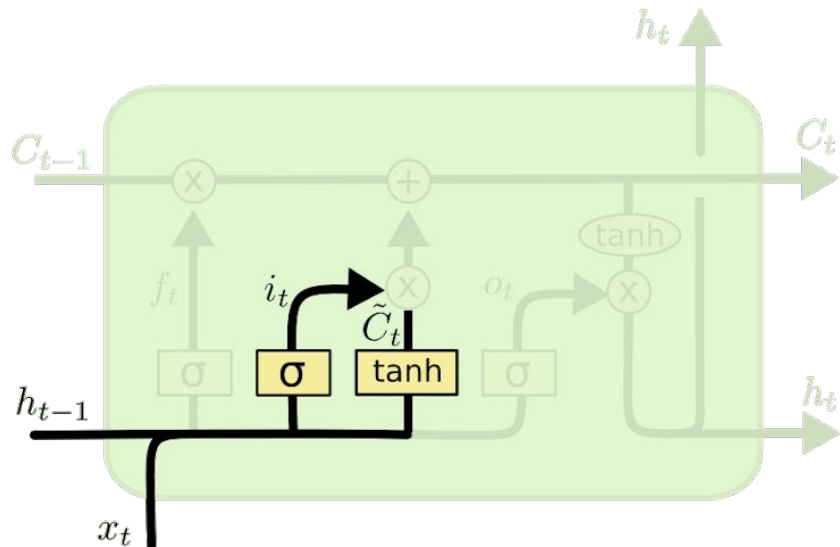


Sau khi tổng hợp thông tin giữa từ hiện tại và hidden state của timestep phía trước, **cổng quên** quyết định **bao nhiêu % thông tin sẽ được giữ lại / loại bỏ** từ cell state phía trước



Cổng đầu vào

Input Gate



Cổng vào quyết định **bao nhiêu % thông tin** sẽ được lấy từ bộ nhớ mới để bổ sung vào cell state.

$$i_t = \sigma(W_i x_t + U_i h_{t-1})$$

$$\tilde{C}_t = \tanh(W_C x_t + U_C h_{t-1})$$

Từ hiện tại Hidden state phía trước

Từ hiện tại Hidden state phía trước

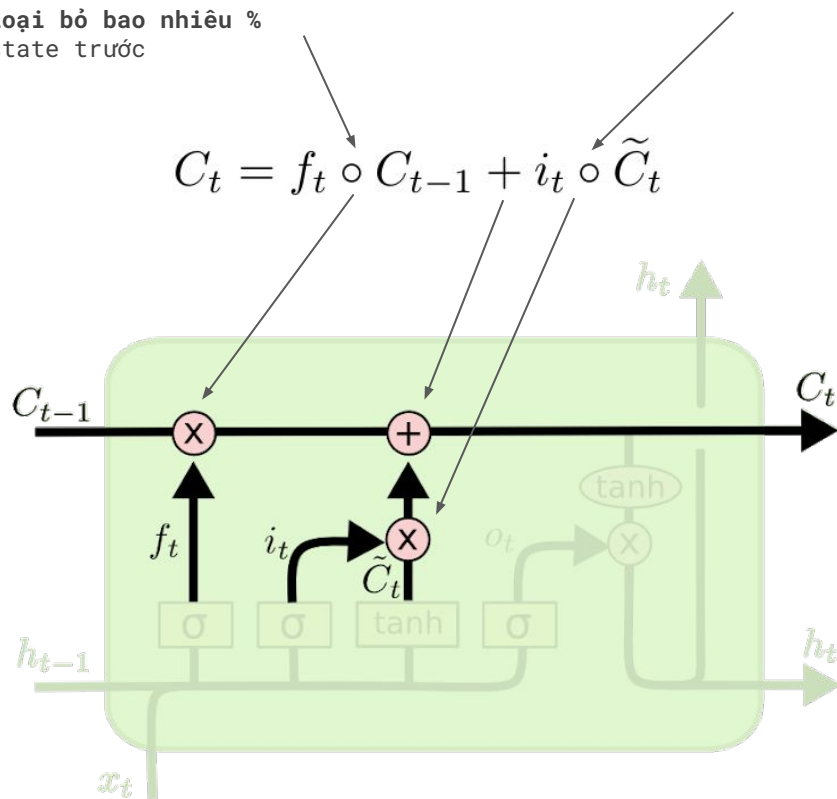
Bộ nhớ mới **chứa đựng** thông tin giữa từ hiện tại và hidden state của timestep phía trước. Đây có thể coi như một mạng nơ ron nhỏ.

Phép Cập nhật

Update Operation

Quyết định **giữ lại/loại bỏ** bao nhiêu %
thông tin của cell state trước

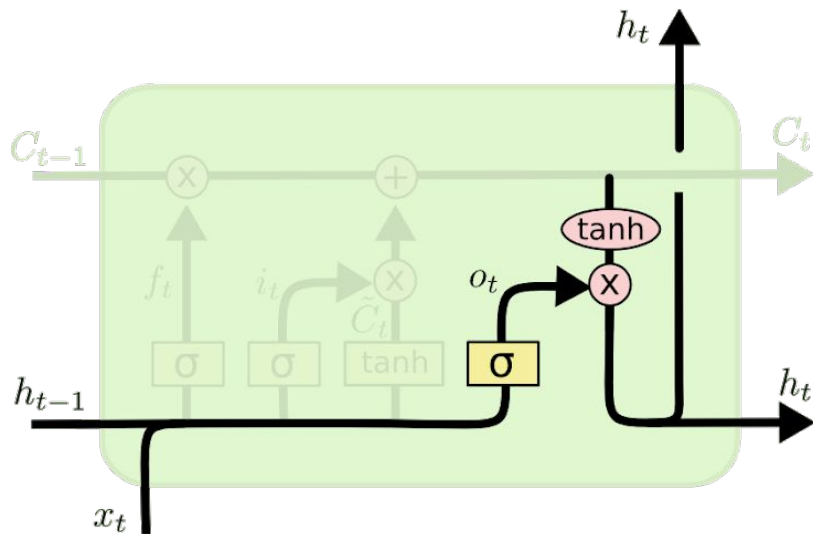
Quyết định bổ sung **bao nhiêu** %
thông tin từ bộ nhớ mới



2 phép nhân trên đều là phép
nhân **Hadamard**

Cổng ra

Output Gate



Cổng ra quyết định bao nhiêu % thông tin sẽ được chuyển từ cell state vào hidden state.

Từ hiện tại Hidden state phía trước

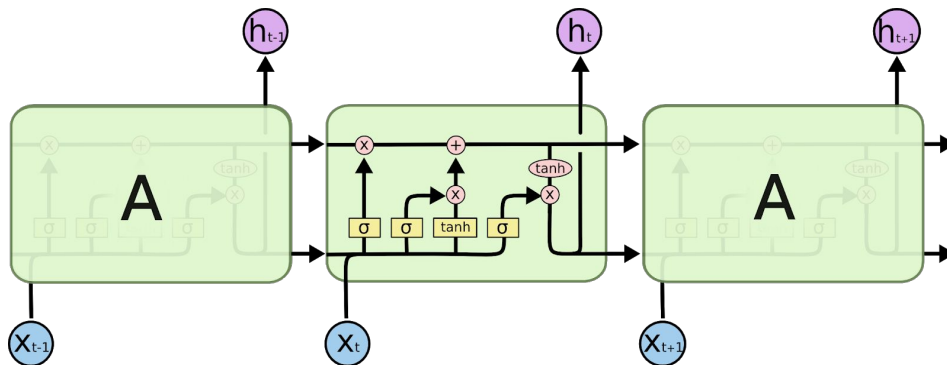
$$o_t = \sigma(W_o x_t + U_o h_{t-1})$$

$$h_t = o_t \circ \tanh(C_t)$$

Cell State và Hidden State sẽ tiếp tục được dùng trong những timestep tiếp theo.

Đưa ra quyết định

Make a decision



$$\begin{aligned}
 f_t &= \sigma(W_f x_t + U_f h_{t-1}) \\
 i_t &= \sigma(W_i x_t + U_i h_{t-1}) \\
 \tilde{C}_t &= \tanh(W_C x_t + U_C h_{t-1}) \\
 C_t &= f_t \circ C_{t-1} + i_t \circ \tilde{C}_t \\
 o_t &= \sigma(W_o x_t + U_o h_{t-1}) \\
 h_t &= o_t \circ \tanh(C_t)
 \end{aligned}$$

Quyết định của các cổng đều phụ thuộc vào hidden state phía trước và từ hiện tại, thông qua giá trị của hàm sigmoid (trong khoảng từ 0 đến 1).

LSTM code

LSTM

$$\begin{aligned}
 f_t &= \sigma(W_f x_t + U_f h_{t-1}) \\
 i_t &= \sigma(W_i x_t + U_i h_{t-1}) \\
 \tilde{C}_t &= \tanh(W_C x_t + U_C h_{t-1}) \\
 C_t &= f_t \circ C_{t-1} + i_t \circ \tilde{C}_t \\
 o_t &= \sigma(W_o x_t + U_o h_{t-1}) \\
 h_t &= o_t \circ \tanh(C_t)
 \end{aligned}$$

```

import tensorflow as tf

# params
W = tf.get_variable('W', shape=[4, state_size, state_size],
                    initializer=xav_init())
U = tf.get_variable('U', shape=[4, state_size, state_size],
                    initializer=xav_init())

def lstm(pre_layer, x):

    pre_h, pre_c = tf.unstack(pre_layer)

    # GATES
    # Input Gate
    i_t = tf.sigmoid(W[0].dot(x) + U[0].dot(pre_h))

    # Forget Gate
    f_t = tf.sigmoid(W[1].dot(x) + U[1].dot(pre_h))

    # Output Gate
    o_t = tf.sigmoid(W[2].dot(x) + U[2].dot(pre_h))

    # New Memory Cell
    n_c_t = tf.tanh(W[3].dot(x) + U[3].dot(pre_h))

    # Final Memory Cell
    c = f_t * pre_c + i_t * n_c_t

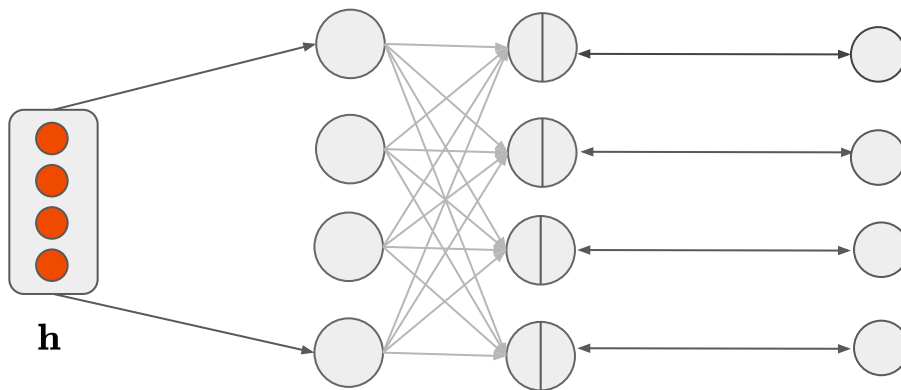
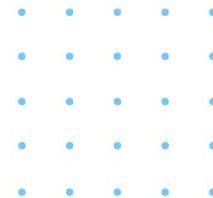
    h = o_t * tf.tanh(c)

    return tf.stack([h, c])

```

Bài toán nhận diện câu

Sentence Classification



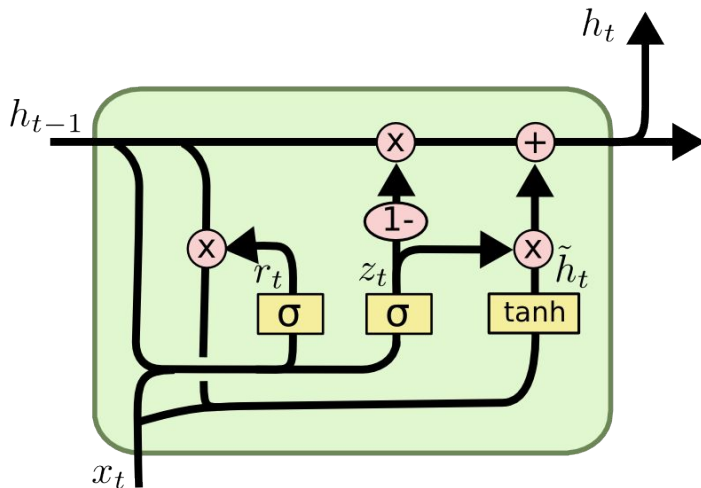
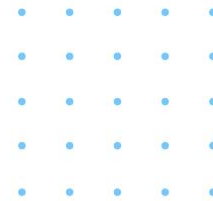
Sử dụng **lịch sử dồn** (vector h) từ các từ phía trước làm đầu vào cho bài toán nhận diện câu

Nếu cell là LSTM thì **vẫn sử dụng hidden state** mà không sử dụng cell state.



Biến thể của LSTM

LSTM Variant



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

GRU (Gated Recurrent Unit) có cơ chế tương tự LSTM tuy nhiên ít tham số hơn và không sử dụng cell state

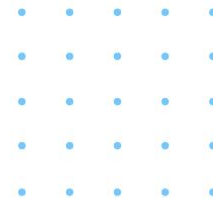


<https://arxiv.org/abs/1406.1078>

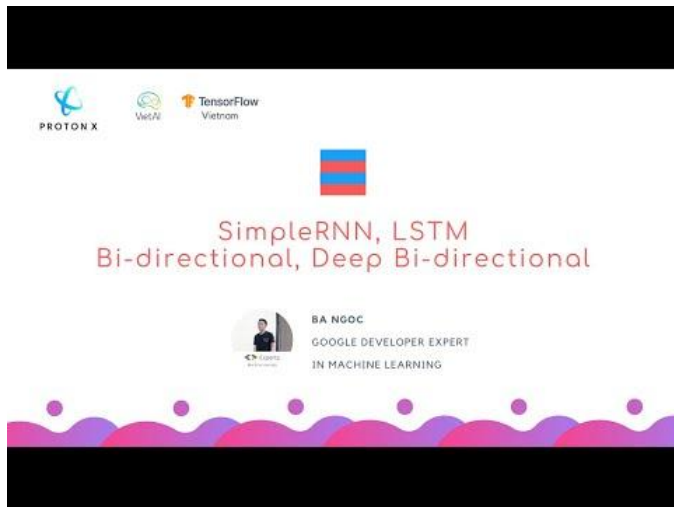
<https://github.com/protonx-tf-03-projects/GRU>

Bài toán nhận diện câu + Sinh từ

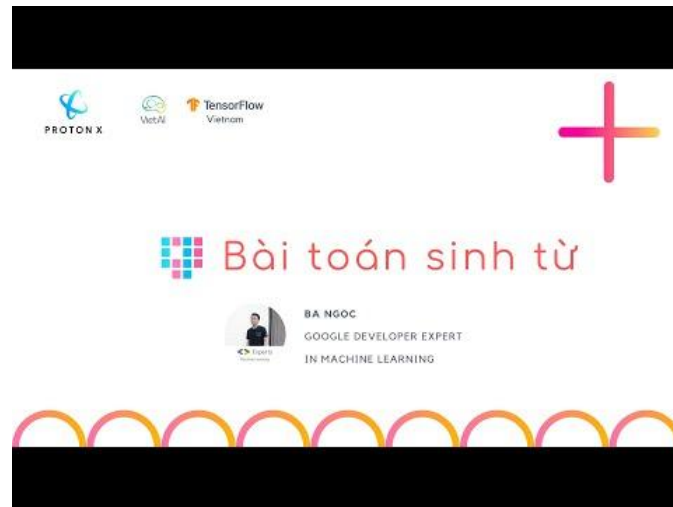
Sentence Classification + Generate Text



Bài toán Nhận diện câu: SimpleRNN, LSTM, Bidirectional
<https://youtu.be/X6bYTZJkEDQ>



Bài toán Sinh từ
<https://www.youtube.com/watch?v=-gyb1dpeGCA>

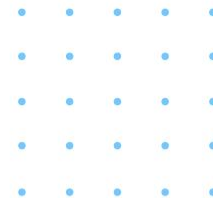


Đến thời điểm hiện tại thì bài toán **phân loại mĩa mai** vẫn đang được sử dụng cho đề thi Tensorflow



Deep Bidirectional

Deep Bidirectional



Deep Bidirectional
<https://youtu.be/Ifnxru7brak>



Tính toán tham số mạng RNN
Deep Bi-directional



BA NGOC
GOOGLE DEVELOPER EXPERT
IN MACHINE LEARNING

